National University of Computer & Emerging Sciences

Lecture 22

Peterson Solution

Peterson's Solution

Combine the previous two algorithms:

```
int turn;
bool interested[2];
```

- **Interested[0] = FALSE**
  - Process 0 is not interested
- **Interested[0] = TRUE**
  - Process 0 is interested
- **Interested[1] = FALSE**
  - Process 1 is not interested
- **Interested[1] = TRUE**
  - Process 1 is interested

Peterson's Algorithm

Shared variables are created and initialized before either process starts.

The shared variables flag[0] and flag[1] are initialized to FALSE because neither process is yet interested in the critical section.

The shared variable turn is set to either 0 or 1 randomly (or it can always be set to say 0).

```
repeat
flag[i] := TRUE;
turn := j;
while (flag[j] and turn=j) do no-op;
CRITICAL SECTION
flag[i] := FALSE;
REMAINDER SECTION
until FALSE;
```

Information common to both processes:

turn = 0

flag[0] = FALSE

flag[1] = FALSE

Peterson's Solution

Process 0

```
while(TRUE)
{ interested[0] = TRUE;
  turn = 0;
  // wait
  while(interested[1] == TRUE && turn == 0 );
  critical_section();
  interested[0] = FALSE;
  noncritical_section();}
```

F && T

Timeout

Process 1

```
while(TRUE)
{
  interested[1] = TRUE;
  turn = 1;
  // wait
  while(interested[0] == TRUE && turn == 1 );
  critical_section();
  interested[1] = FALSE;
  noncritical_section();
  }
```

F && T

Peterson's Solution

Process 0

```
while(TRUE)
{ interested[0] = TRUE;
  turn = 0;

    // wait
  while(interested[1] && turn == 0 );

  critical_section();
  interested[0] = FALSE;
  noncritical_section();}
```

T && F

Process 1

```
while(TRUE)
{
  interested[1] = TRUE;
  turn = 1;
  // wait
  while(interested[0] && turn == 1 );

  critical_section();
  interested[1] = FALSE;
  noncritical_section();
  }
```

F && T

Timeout

Can not be TRUE at the same time.
Thus used to break tie

Proof of three properties

- **Mutual exclusion is preserved since:**
  - P0 and P1 are both in CS only if flag[0] = flag[1] = true and only if turn = i for each Pi (impossible)
- **We now prove that the progress and bounded waiting requirements are satisfied:**
  - Pi cannot enter CS only if stuck in while() with condition flag[j] = true and turn = j.
  - If Pj is not ready to enter CS then flag[j] = false and Pi can then enter its CS
  - If Pj has set flag[j]=true and is in its while(), then either turn=i or turn=j
  - If turn=i, then Pi enters CS. If turn=j then Pj enters CS but will then reset flag[j]=false on exit: allowing Pi to enter CS
  - but if Pj has time to reset flag[j]=true, it must also set turn=i

Peterson solution Dry Run1

EXAMPLE 1	
Process 0	Process 1
i = 0, j = 1	i = 1, j = 0
flag[0] := TRUE turn := 1 check (flag[1] = TRUE and turn = 1) - Condition is false because flag[1] = FALSE - Since condition is false, no waiting in while loop - Enter the critical section - Process 0 happens to lose the processor	
	flag[1] := TRUE turn := 0 check (flag[0] = TRUE and turn = 0) - Since condition is true, it keeps busy waiting until it loses the processor
- Process 0 resumes and continues until it finishes in the critical section - Leave critical section flag[0] := FALSE - Start executing the remainder (anything else a process does besides using the critical section) - Process 0 happens to lose the processor	
	check (flag[0] = TRUE and turn = 0) - This condition fails because flag[0] = FALSE - No more busy waiting - Enter the critical section

Peterson solution Dry Run2

EXAMPLE 2	
Process 0	Process 1
i=0, j=1	i=1, j=0
flag[0] = TRUE turn = 1 - Lose processor here	
	flag[1] := TRUE turn := 0 check (flag[0] = TRUE and turn = 0) - Condition is true so Process 1 busy waits until it loses the processor
check (flag[1] = TRUE and turn = 1) - This condition is false because turn = 0 - No waiting in loop - Enters critical section	

Peterson solution Dry Run3

EXAMPLE 3	
Process 0	Process 1
i=0, j=1	i=1, j=0
flag[0] = TRUE - Lose processor here	
	flag[1] = TRUE turn = 0 check (flag[0] = TRUE and turn = 0) - Condition is true so, Process 1 busy waits until it loses the processor
turn := 1 check (flag[1] = TRUE and turn = 1) - Condition is true so Process 0 busy waits until it loses the processor	
	check (flag[0] = TRUE and turn = 0) - The condition is false so, Process 1 enters the critical section